

Assignment 7

This exercise sheet is dedicated to polynomial interpolation.

Graded exercise

Exercise 25 (Interpolating oscillatory functions)

In this exercise, we implement polynomial interpolation and then we analyze its performance for oscillatory functions. For didactic purposes, to solve this exercise you are **not** allowed to use `scipy` routines.

- (a) Write a routine `Lagrange_pol` that, given in input an index $i \in \{0, \dots, m\}$, an array `x_pts` of $m + 1$ interpolation points $x_0, \dots, x_m$, and an array `x_eval` of evaluation points, returns the values, at the evaluation points, of the i -th Lagrange polynomial $L_i(x)$ associated to the given interpolation points.
- (b) Write a routine `pol_interp` that, given in input two arrays corresponding to data points (x_i, f_i) , $i = 0, \dots, m$ (one array for the x -coordinates and one array for the corresponding function values) and an array `x_eval` of evaluation points, returns the values, at the evaluation points, of the interpolating polynomial associated to the data points.

Hint: Use the routine from the previous task.

Remark: As mentioned in the lecture notes and in class, the Lagrange interpolation formula is not the most efficient computationally for evaluating the polynomial. However, it is fine to use it here for didactic purposes.

- (c) Consider $f(x) = \sin(2k\pi x)$, $x \in [-1, 1]$, and set, for the moment, $k = 1$. Write a script that:
 - using the routine from the previous task, builds the interpolant of f using $m + 1$, $m = 16$, equispaced points in $[-1, 1]$ (including the extremes) and evaluates it on $n = 1001$ equispaced points in $[-1, 1]$;
 - does the same but with Chebyshev nodes for interpolation (for evaluating the polynomial, you can still use $n = 1001$ equispaced points);

- produces a plot with the function f , its interpolant using equispaced nodes and its interpolant using Chebyshev nodes.

When handing in the assignment, please remember to include both the code and the plot.

- (d) Repeat (c) but now with $k = 4$ and $m = 32$ (you don't have to create a different script, you can just comment the parameters in (c) and insert the new ones, but remember to hand in the new plot). How do the performances of equidistant and Chebyshev nodes compare? Explain why the two sets of nodes perform differently and why this was not observable with $k = 1$ as in (c).

Other exercises

Exercise 26 (Interpolation error)

- (a) We consider the interval $I = [0, 1]$ and the interpolation points $x_0 = 0$, $x_1 = \frac{1}{4}$, $x_2 = \frac{1}{2}$ and $x_3 = \frac{3}{4}$. Compute (by hand) the polynomial P of degree 3 that interpolates $f(x) = \sin(2\pi x)$ at the given interpolation points.
- (b) Write the monic polynomial $\omega(x)$ associated to the interpolation points $x_0, \dots, x_3$. Show that $\|\omega(x)\|_\infty = \frac{1}{256}$ and then use this to provide a bound on the interpolation error $\|f - P\|_\infty$ on I .
- (c) For a given set of interpolation points $x_0 < x_1 < \dots < x_{2020}$ on the interval $I = [0, 2020]$, write the interpolation polynomial of degree 2020 for the function

$$f(x) = \sum_{k=1}^{2020} \frac{1}{(k^2 + k)k!} x^k.$$

Exercise 27 (Horner's scheme)

Horner's scheme gives an efficient algorithm to evaluate a polynomial

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

at a given point x_0 . It is based on the following representation for the polynomial:

$$p(x) = (((((a_n x + a_{n-1})x + a_{n-2})x + a_{n-3})x + \dots)x + a_1)x + a_0,$$

which can be computed from the inner and to the outer factors.

- (a) How many multiplications and sums are needed by a standard polynomial evaluation at x_0 (computation of all powers of x_0 , the multiplication by the coefficients and final sum)?

- (b) How many multiplications and sums are needed to evaluate the polynomial using Horner's scheme?
- (c) Take the polynomial $p(x) = 2x^4 + 1x^3 - 3x + 2$, and evaluate it at $x_0 = 2$ using Horner's scheme (by hand).